No Collaborators

Final Project

Goal: For all Pokemon, I wanted to see how pokemon compared to each other based on their stats and what pokemon would be considered about the same strength. To analyze this, I clustered based on basic stats (hp, attack, defense, speed) and special stats (special attack, special defense). I decided on having 4 clusters for basic and special stats, the clusters labels that I decided for basic stats was “weak, average, decent, strong” and for special stats I labeled them “weak, strong defense only, strong attack only, strong”. In addition, I also computed rank which was based on each pokémon’s total stat in comparison to all pokemon and I also printed which group the pokemon was part of “Baby Pokemon, Ordinary, Legendary, Mythical, Ultra Beast, Paradox, Fossil”.

Dataset: I used a pokemon dataset from Kaggle which consisted of all 1025 pokemon. https://www.kaggle.com/datasets/guavocado/pokemon-stats-1025-pokemons/code

Loading/Cleaning: I loaded this dataset into rust just as a csv by openng and reading the file. I didn’t clean the dataset but instead I only extracted the values that I needed and stored it into my Pokemon struct. I just didn’t interact with the information that I wasn’t using.

Modules:

- Pokemon:
    - Reading the pokemon csv file
    - Loading into my pokemon struct
    - Normalizing the stat vectors
- Clustering:
    - Kmeans clustering, determining clusters and centroids
    - Centroid labeling
    - Cluster mapping and getting cluster for pokemon
    - Printing info, user input, finding rank and group

Key Type:

- Pokemon struct:
    - used for making each pokemon based on the values that I wanted to use for my program, stores the data from the csv under each corresponding pokemon
    - inputted: name, hp, attack, defense, speed, special attack, special defense, total, group

Main Workflow:

No Collaborators

- Essentially just had a pokemon focused module and a clustering module to help with organization and sorting out my code making it easier to read and to fix any errors. The main function takes care of all the important calls to functions and the two modules take care of more details
- Main first works with pokemon module for set up and then goes through the functions in the clustering module and lastly tests are at the bottom of the main file.
- These all interact to produce the clustering that I wanted to achieve and also other information I wanted to present to the user.

Tests: first I made some fake pokemon that I initialized with name, stats and group

- test_normalized_stats()
    - this is to make sure that my normalized stats function works correctly by making sure that all my stats are kept after normalization and also that the normalization is done correctly by making sure that the resulting numbers are between 0 and 1
    - this is important because it makes sure that my stats are being normalized correctly so that I can cluster them, if they aren't then I wont be able to cluster them correctly
- test_clustering()
    - this makes sure that the the correct number of clusters was made and that it stores the correct number of centroids
    - this is important because I need to make sure that my clusters function is actually generating the correct number of clusters and that there is the correct number of centroids so that I can meet my goal of clustering my pokemon and labelling the clusters appropriately
- test_labelling()
    - this makes sure that the centroids of clusters are being labeled correctly, by having sample centroids and make sure that my function assigns them the correct label according to the rules I have set
    - this is important because I need to be able to interpret my clustering correctly and it would be bad if my strong cluster was getting labeled weak, so this is to make sure I am labelling the correct stats with the correct title
- test_cluster_map()
    - this makes sure that my cluster map is working properly by mapping the pokemon with the right label
    - this is important because it would be bad if I outputted the wrong label for a pokemon when instead it belongs in a different cluster, so that I can accurately show my results of clustering to the user

No Collaborators

```
running 4 tests
test tests::test_cluster_map ... ok
test tests::test_labelling ... ok
test tests::test_normalized_stats ... ok
test tests::test_clustering ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.01s
```

Results:

- specific output depends on what pokemon is entered
- this output is for the example of entering “Arceus”
- but the cluster information before user input should be the sane and what is printed for basic stats, special stats, rank and group will vary depending on what pokemon is entered
- from this output you can see the 4 clusters that my program generated, the labels for them and how many pokemon are part of which cluster and then which cluster the pokemon entered falls into

```
basic stats:
   cluster 1 - decent (240 pokemon)
   cluster 2 - strong (242 pokemon)
   cluster 3 - weak (333 pokemon)
   cluster 4 - average (210 pokemon)
special stats:
   cluster 1 - strong attack only (243 pokemon)
   cluster 2 - strong defense only (220 pokemon)
   cluster 3 - weak (411 pokemon)
   cluster 4 - strong (151 pokemon)
enter a pokemon name with first letter capitalized
Arceus
basic stats: strong
special stats: strong
rank #: 1/1025
group: Mythical
```

Usage Instructions:

- Run the code, enter a pokemon name with the first letter capitalized and everything else will be printed automatically
- Expected runtime about 6 seconds

Citation:

- https://rust-ml.github.io/book/3_kmeans.html
- Used this website to help with implementing k mean clustering in rust, it shows how to use linfa and so I followed the basic structure and steps to make kmeans clustering work for my code,